

Routing Strategies for Ring and General Topologies in OMNeT++

Gerbaudo, Nicolás Ignacio; Jorge González, Nicolás; Ontivero, Nahuel Mauricio; Vides, Alejo Miguel
Grupo 25, Facultad de Matemática, Astronomía, Física y Computación (FaMAF)
Universidad Nacional de Córdoba, Argentina

Abstract

Write a concise abstract in English (150–250 words). Summarize the simulated network scenarios, the routing algorithm you designed, the main performance metrics (delay, hop count, buffer/link utilization, stability), and the key findings when comparing your solution against the kickstarter baseline. Mention whether routing loops were observed and, if applicable, how your algorithm generalizes to arbitrary topologies.

Index Terms— Network simulation, OMNeT++, routing, ring topology, performance evaluation, queueing, stability.

1 Introducción

Este artículo aborda el problema del enrutamiento en una red en anillo de 8 nodos simulada en OMNeT++. Cada nodo cuenta con una capa de aplicación (**app**), una capa de red (**net**) y dos capas de enlace (**lnk[0]** y **lnk[1]**), que conectan al nodo con sus dos vecinos. El objetivo es analizar el comportamiento del algoritmo de enrutamiento provisto por la cátedra, identificar sus limitaciones y evaluar el rendimiento del sistema bajo distintos escenarios de tráfico.

En el algoritmo del kickstarter, cada nodo evalúa si es el destino final del paquete recibido; de serlo, lo entrega a su capa de aplicación; de lo contrario, lo reenvía siempre por la interfaz **toLnk[0]**, sin considerar la distancia ni el estado de la red, lo que implica que el tráfico siempre se envía en sentido horario a lo largo del anillo hasta llegar al destino. En consecuencia, la mitad de la capacidad de la red queda sin utilizar y ciertos enlaces concentran todo el tráfico, mientras otros permanecen inactivos; así se genera saturación en los buffers y demoras crecientes. En este trabajo evaluamos el rendimiento del sistema mediante métricas de demora extremo a extremo, ocupación de buffers, utilización de enlaces, cantidad de saltos por paquete y paquetes en tránsito por nodo.

2 Escenario y modelo de simulación

2.1 Topología y configuración de tráfico

La red simulada consiste en un anillo de 8 nodos (**node[0]** a **node[7]**), donde cada nodo se conecta con sus dos vecinos mediante enlaces *full-duplex* de 1 Mbps y 100 μ s de retardo. Cada nodo cuenta con tres capas:

../../../../figuras/topologia-red.png

Figure 1: Topología en anillo de 8 nodos.

- una capa de aplicación (**app**), que genera y recibe paquetes;
- una capa de red (**net**), que decide por qué interfaz reenviar cada paquete; y
- dos capas de enlace (**lnk[0]** y **lnk[1]**), cada una conectada a uno de los vecinos.

La topología de la red y la estructura interna de un nodo se muestran en las Figs. ?? y ??, respectivamente.

../../../../figuras/topologia-nodo-0.png

Figure 2: Estructura interna de `node[0]`: `lnk[0]` conecta con `node[7]` y `lnk[1]` conecta con `node[1]`.

Table 1: Configuración experimental por escenario.

Parámetro	Caso 1	Caso 2	Comp.
Fuentes activas	<code>n[0]</code> , <code>n[2]</code>	<code>n[0-4]</code> , <code>n[6-7]</code>	1 y 2
Destino	<code>n[5]</code>	<code>n[5]</code>	idem
<code>packetByteSize</code>	125000	125000	[val.]
<code>interArrivalTime</code>	<code>exp(1)</code>	<code>exp(0.5-10)</code>	[val.]
T. simulación (s)	200	200	[val.]
Semilla	0 (default)	0 (default)	[val.]

2.2 Métricas registradas

Las métricas se instrumentaron directamente en los módulos del modelo:

- **Demora extremo a extremo:** registrada en `app` al recibir cada paquete, como la diferencia entre el tiempo actual y el tiempo de creación. Se registra como vector y como escalar (promedio final).
- **Salto por paquete:** registrados en `app` al recibir cada paquete, leyendo el campo `hopCount` incrementado en `net` en cada reenvío. Se registra como vector y como escalar (promedio final).
- **Ocupación de buffers:** registrada en `lnk` como vector cada vez que el tamaño de la cola cambia.
- **Utilización de enlaces:** acumulada en `lnk` sumando el `serviceTime` de cada paquete transmitido. Se reporta como escalar al final dividiendo por el tiempo total de simulación.

- **Paquetes en tránsito por nodo:** contados en `net` como escalar, incrementando un contador cada vez que el nodo reenvía un paquete ajeno.

Table 2: Métricas instrumentadas en el simulador.

Métrica	Módulo	Tipo	Unid.
Demora E2E	<code>app</code>	vector + escalar	s
Salto / paquete	<code>app</code>	vector + escalar	adim.
Ocupación buffer	<code>lnk</code>	vector	pkts
Utilización enlace	<code>lnk</code>	escalar	%
Paquetes en tránsito	<code>net</code>	escalar	adim.

3 Tarea de análisis: evaluación del kickstarter

3.1 Caso 1: tráfico configurado (`node[0]` y `node[2]` → `node[5]`)

Configuración experimental. Sólo `node[0]` y `node[2]` transmiten hacia `node[5]` con `interArrivalTime` distribuido exponencialmente con media de 1 segundo y paquetes de 125000 bytes. El resto de los nodos no genera tráfico. La simulación corre durante 200 segundos.

Métricas obtenidas. En este caso, obtuvimos las siguientes métricas:

- **Demora extremo a extremo:** tiempo transcurrido desde que un paquete se crea en la `app` origen hasta que se recibe en la `app` destino.
- **Cantidad de saltos por paquete:** número de saltos que realiza un paquete desde su origen hasta su destino, incrementado en cada reenvío por `net`.
- **Ocupación de buffers:** cantidad de paquetes en espera en la cola de cada `lnk` a lo largo del tiempo.
- **Utilización de enlaces:** fracción del tiempo de simulación durante la cual cada `lnk` estuvo transmitiendo activamente.
- **Paquetes en tránsito por nodo:** cantidad total de paquetes ajenos que cada `net` reenvió durante la simulación.

Uso de recursos. Los recursos de la red se utilizan de forma muy desigual. Dado que el algoritmo del kickstarter reenvía todos los paquetes por `lnk[0]` (sentido horario), sólo 5 de las 8 interfaces `lnk[0]` transportan paquetes, mientras que todas las interfaces `lnk[1]` permanecen inactivas.

El enlace más cargado es `node[0].lnk[0]`, con una utilización del 99.5% y una ocupación media de buffer

de 92.92 paquetes (Tabla ??), lo que indica saturación sostenida. Esto se debe a que por este enlace pasan los paquetes generados por `node[0]` y `node[2]`. La Fig. ?? confirma que el buffer de `node[0].lnk[0]` crece de forma sostenida a lo largo de la simulación, mientras que el de `node[2].lnk[0]` se mantiene relativamente acotado.

Los nodos intermediarios `node[1]`, `node[6]` y `node[7]` muestran una ocupación de buffer máxima de 1 paquete (Fig. ??), a pesar de utilizar sus enlaces entre el 93.5% y el 99%. Esto refleja que los paquetes fluyen de forma casi continua pero sin acumulación significativa en esos nodos.

En cuanto a los paquetes reenviados (Fig. ??), `node[6]` y `node[7]` reenvían aproximadamente 197 y 198 paquetes respectivamente. Dado que ambas fuentes generan aproximadamente 200 paquetes cada una y ambos nodos están en la ruta de las dos fuentes, se esperaría que reenviaran unos 400 paquetes. El hecho de que sólo hayan reenviado unos 200 es evidencia adicional de saturación: gran parte de los paquetes quedaron atrapados en los buffers al finalizar la simulación.

Table 3: Resultados del Caso 1 (baseline kickstarter).

Métrica	Prom.	Desv.
Demora extremo a extremo (s)	51.16	—
Salto por paquete	3.92	—
<i>Utilización de enlace (%)</i>		
<code>node[0].lnk[0]</code>	99.5	—
<code>node[1].lnk[0]</code>	93.5	—
<code>node[2].lnk[0]</code>	94.0	—
<code>node[6].lnk[0]</code>	98.5	—
<code>node[7].lnk[0]</code>	99.0	—
<i>Ocupación de buffer (paquetes)</i>		
<code>node[0].lnk[0]</code>	92.92	54.83
<code>node[1].lnk[0]</code>	0.50	0.50
<code>node[2].lnk[0]</code>	3.29	2.51
<code>node[6].lnk[0]</code>	0.50	0.50
<code>node[7].lnk[0]</code>	0.50	0.50
<i>Paquetes reenviados por nodo</i>		
<code>node[0]</code>	186	—
<code>node[1]</code>	187	—
<code>node[6]</code>	197	—
<code>node[7]</code>	198	—

El resto de los enlaces tiene utilización 0%.

Los `node[3].lnk[0]`, `node[4].lnk[0]` y `node[5].lnk[0]` no registraron ocupación de buffer por no haber tenido tráfico en sus colas durante la simulación.

El resto de los nodos reenvió 0 paquetes.

../../../../figuras/caso1_buffers_fuentes.png

Figure 3: Ocupación de buffers a lo largo del tiempo — nodos fuente (`node[0]` y `node[2]`).

../../../../figuras/caso1_buffers_intermediarios.png

Figure 4: Ocupación máxima de buffers — nodos intermediarios (`node[1]`, `node[6]` y `node[7]`).

Possibilidades de mejora. El rendimiento puede mejorarse. El algoritmo actual ignora que cada nodo dispone de dos interfaces y que existen dos caminos posibles hacia cualquier destino. Al forzar siempre el uso de `lnk[0]`, se concentra todo el tráfico en un subconjunto de enlaces, dejando la mitad de la capacidad de la red completamente sin utilizar. Esto genera sat-



Figure 5: Demora extremo a extremo a lo largo del tiempo — node[5].



Figure 6: Paquetes reenviados por nodo durante la simulación.

3.2 Caso 2: tráfico sostenido hacia node[5]

A partir de `interArrivalTime` = 7s la red permanece estable porque los buffers de todos los nodos se mantienen acotados a lo largo de la simulación (Figs. ??-?? y Tabla ??). Para valores menores de `interArrivalTime`, al menos un buffer crece indefinidamente, lo que evidencia saturación.

La estabilización de la red a partir de `iat`=7 también se refleja en la demora media (Fig. ??), que cae drásticamente de 34s a 7s entre `iat`=5 y `iat`=7, y se mantiene en valores bajos para valores mayores de `iat`.

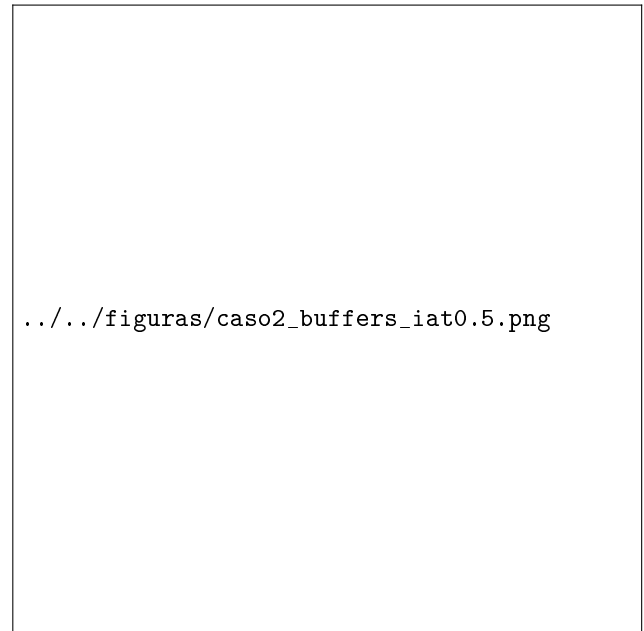


Figure 7: Ocupación de buffers a lo largo del tiempo — `iat`=0.5 s.

Table 4: Exploración del Caso 2: métricas vs. `interArrivalTime`.

<code>interArriv.</code>	Demora (s)	Ocup. buffer máx.	Estab.
0.5	73.19	410	No
1.0	65.92	227	No
2.0	61.43	124	No
3.0	59.25	70	No
5.0	34.01	40	No
7.0	7.16	4	Sí
8.0	5.56	4	Sí
10.0	4.71	2	Sí

uración en los enlaces más transitados y demoras crecientes, como se observa en la Fig. ??.

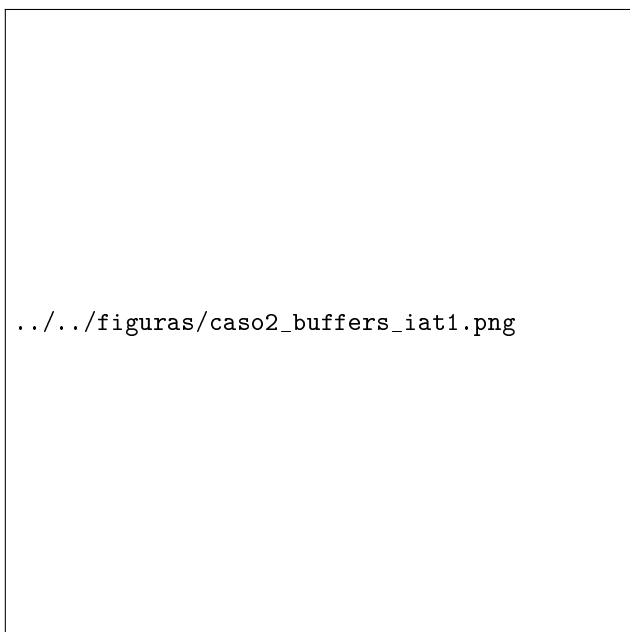


Figure 8: Ocupación de buffers a lo largo del tiempo
— $\text{iat}=1\text{s}$.

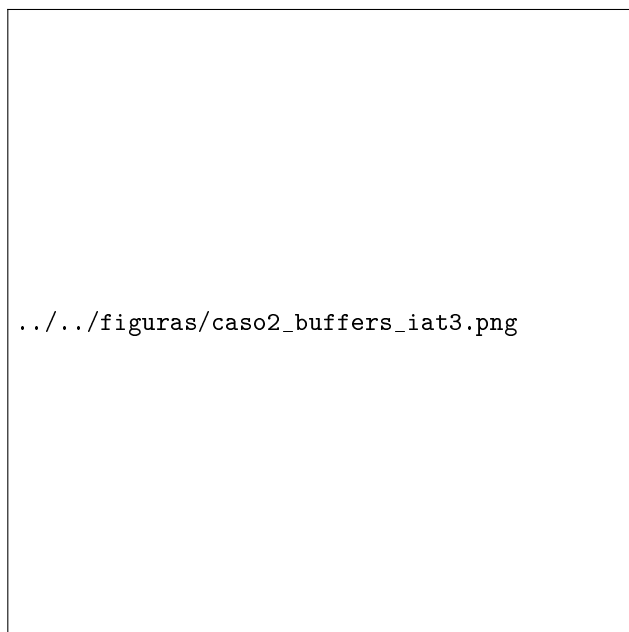


Figure 10: Ocupación de buffers a lo largo del tiempo
— $\text{iat}=3\text{s}$.

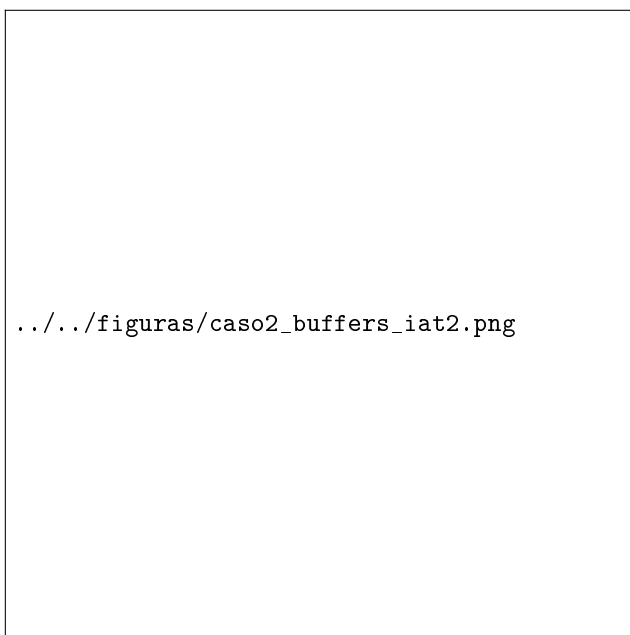


Figure 9: Ocupación de buffers a lo largo del tiempo
— $\text{iat}=2\text{s}$.



Figure 11: Ocupación de buffers a lo largo del tiempo
— $\text{iat}=5\text{s}$.



Figure 12: Demora media vs. `interArrivalTime` —
Caso 2.

4 Diseño del algoritmo de enrutamiento propuesto

Guía para completar: Corresponde a la **Tarea de Diseño**. Explique qué información usa cada nodo, cómo la obtiene (p. ej. paquetes hello/echo, tablas locales) y cómo decide la interfaz de salida. Describa decisiones de diseño, limitaciones y por qué su enfoque supera al baseline. Puede usar pseudocódigo, diagrama de flujo o tabla de reglas. Recuerde: no use `cTopology` ni iteradores de `cGate` para descubrir vecinos.

4.1 Información disponible y estructuras de datos

[Describa tablas de rutas, métricas locales, mensajes de control, etc.]

4.2 Reglas de decisión

Guía para completar: Presente el algoritmo en tres niveles: (1) descripción en prosa, (2) tabla de reglas o pseudocódigo, (3) ejemplo numérico con un paquete concreto que atraviesa varios nodos. Si intercambian mensajes de control, muestre el formato del paquete hello/echo en una tabla.

[Detalle el algoritmo. Ejemplo de pseudocódigo:]

Table 5: Ejemplo de tabla de rutas locales en un nodo (complete con su diseño).

Destino	Interfaz	Saltos
node[5]	lnk[1]	2
node[0]	lnk[0]	3
node[2]	lnk[0]	1
[...]	[...]	[...]

```
al recibir paquete p en interfaz i:
  si dest(p) == id_local:
    entregar a capa app
  sino:
    j <- elegir_interfaz(p, tablas_locales)
    reenviar p por interfaz j
```

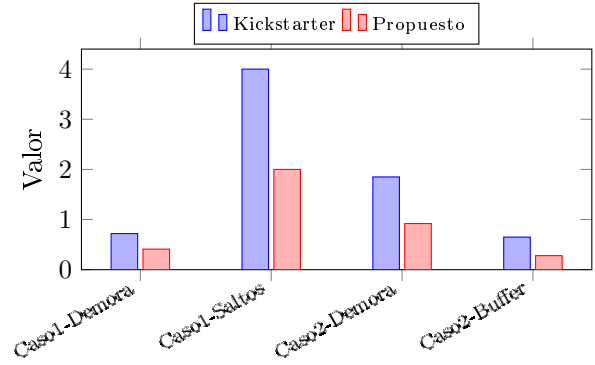


Figure 13: Ejemplo de barras agrupadas para comparar algoritmos. Todo gráfico debe tener ejes etiquetados, unidades y leyenda.

5 Evaluación comparativa

Guía para completar: Compare su algoritmo contra el kickstarter en los Casos 1 y 2 (y en otros escenarios si lo desea). Use tablas side-by-side, gráficos de barras agrupados o boxplots de demora. Cuantifique la mejora (% , factor, absoluto). Responda: ¿cuánto mejoran las métricas? ¿por qué? ¿hubo bucles de enrutamiento? ¿Qué pasaría si la cantidad de nodos es mucho mayor y como funcionaria con respecto al algoritmo propuesto por la cátedra? ¿Qué sucedería si se cae un nodo serían tolerantes a esa falla los algoritmos? Si ajustó parámetros según la nota al pie 4 del enunciado, indíquelo claramente en la metodología.

5.1 Metodología de comparación

[Describa bajo qué condiciones ejecutó cada algoritmo, si usó los mismos escenarios, semillas y parámetros, y cómo calculó promedios o intervalos. Ejemplo: “Para cada *interArrivalTime* del Caso 2 se ejecutaron 5 repeticiones con semillas 1–5; reportamos la media y el máximo de demora extremo a extremo.”]

5.2 Resultados cuantitativos

5.3 Discusión de resultados

Guía para completar: Interprete las tablas y figuras anteriores. Relacione mejoras con decisiones concretas de su diseño (p. ej. balanceo de carga, ruta más corta). Mencione mejoras posibles no implementadas por limitaciones de tiempo o alcance.

[Discusión basada en evidencia. ¿Se observaron bucles? ¿En qué condiciones? ¿Cómo los detectó o evitó?]

Table 6: Comparación baseline vs. algoritmo propuesto (complete con sus datos).

Escenario / Métrica	Kickstarter		Propuesto	
	Media	Máx.	Media	Máx.
Caso 1 — Demora (s)	0.00	0.00	0.00	0.00
Caso 1 — Saltos	0.0	0	0.0	0
Caso 2 — Demora (s)	0.00	0.00	0.00	0.00
Caso 2 — Pérdidas	0	—	0	—

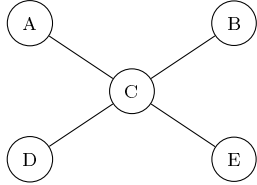


Figure 14: Ejemplo esquemático de topología estrella (`networkStar`). Reemplace por su figura y describa cómo se adapta el algoritmo a N interfaces.

3. Guarde los CSV en `figuras/datos/` y documente en el paper la configuración (`omnetpp.ini`, tiempo de simulación, semilla).

Guía para completar: Ejemplo de flujo de trabajo: (1) simular $\rightarrow$ (2) exportar `.sca` y `.vec` $\rightarrow$ (3) procesar con script propio o planilla $\rightarrow$ (4) generar PDF/PNG $\rightarrow$ (5) incluir con `\includegraphics`. Mantenga los scripts en el repositorio junto al paper para reproducibilidad.

6 Tarea estrella (opcional)

Guía para completar: Si realizaron la tarea opcional, documente la generalización a topologías arbitrarias y N interfaces (p. ej. `networkStar`). Explique cómo detectan interfaces activas y gates desconectadas. Incluya al menos un experimento adicional con figura o tabla. Si no la realizaron, elimine esta sección o indique brevemente por qué quedó fuera de alcance.

[Contenido opcional: topología estrella, desafíos de implementación, resultados comparativos con el anillo.]

A Guía para tablas y gráficos con datos de OMNeT++

Guía para completar: Este apéndice es **solo orientativo**: Les tiene que servir para saber cómo pasar de la simulación a evidencia publicable. Pueden eliminarlo en la entrega final.

A.1 Exportar resultados

1. Ejecute cada escenario con **semillas distintas** o repeticiones suficientes si busca intervalos de confianza.
2. Exporte **escalares** (promedio, máximo, conteo) y **vectores** (series temporales) desde el IDE o con `scavetool`.

A.2 Ejemplo de tabla de mejoras porcentuales

Cuando compare algoritmos, una tabla de mejoras complementa la Tab. ??:

Table 7: Mejora relativa del algoritmo propuesto vs. kickstarter (ejemplo).

Métrica (escenario)	Base	Mej. (%)
Demora media (Caso 1)	0.82 s	−43%
Saltos promedio (Caso 1)	4.0	−50%
Demora media (Caso 2)	1.85 s	−50%
Ocup. máx. buffer (Caso 2)	25 pkts	−80%

Guía para completar: Calcule mejora como $(\text{baseline} - \text{propuesto}) / \text{baseline} \times 100$ para métricas donde **menor es mejor** (demora, saltos, pérdidas). Para throughput, invierta la fórmula. Explique en el texto si un −43% en demora es significativo en términos absolutos.

A.3 Cuándo usar cada recurso gráfico

A.4 Incluir figuras externas en L^AT_EX

Para gráficos generados con Python, R, Matplotlib o el IDE de OMNeT++:

```

\begin{figure}[!t]
  \centering
  \includegraphics[width=\linewidth]{figuras/mi-grafico.pdf}
  \caption{Demora media por escenario.}
  \label{fig:mi-grafico}
\end{figure}

```


Table 8: Recursos recomendados según el tipo de evidencia.

Objetivo	Recurso	Ejemplo
Comparar escenarios	Tabla + barras	Tab. ??
Sensibilidad param.	Curva x vs. métrica	Fig. ??
Evolución temporal	Serie de tiempo	Fig. ??
Distrib. demoras	Histograma	[si aplica]
Topología	Diagrama de red	Fig. ??

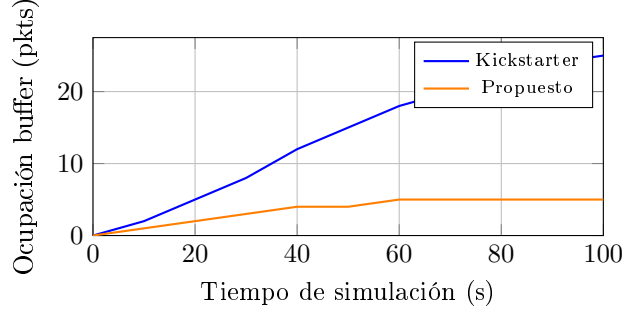


Figure 15: Ejemplo de serie temporal: ocupación de buffer. Útil para mostrar saturación o estabilidad en el Caso 2.

Guía para completar: Prefiera PDF o PNG de alta resolución. En el texto, **siempre** referencie la figura **antes** de que aparezca y explique qué se ve: no basta con insertar el gráfico sin interpretación.

B Checklist de entrega del paper

Guía para completar: Revise este listado antes de subir al repositorio. Borre el apéndice si ya no lo necesita.

- Título, autores, *abstract* e *index terms* en inglés.
- Cuerpo del artículo en español, sin cajas \guia{ }.
- Respuestas explícitas a Caso 1, Caso 2 y evaluación comparativa.
- Al menos **3 figuras** y **2 tablas** con datos propios (no solo los ejemplos de la plantilla).
- Cada conclusión respaldada por una tabla o figura citada.
- Descripción del algoritmo, decisiones de diseño y detección de bucles.
- Extensión final entre 6 y 8 páginas en formato de dos columnas.
- Figuras con ejes, unidades, leyenda y caption descriptivo.

- Bibliografía en formato consistente (IEEE-like o APA).

B.1 Mapa entre tareas del laboratorio y secciones del paper

Table 9: Correspondencia entre el enunciado y las secciones de este documento.

Tarea / pregunta	Sección del paper
Instrumentar métricas	Sec. ??, Tab. ??
Caso 1: métricas y recursos	Sec. ??, Tab. ??
Caso 2: estabilidad	Sec. ??, Fig. ??
Diseño del algoritmo	Sec. ??
Comparación y bucles	Sec. ??
Tarea estrella (opcional)	Sec. ??

C Conclusiones

Guía para completar: Sintetice en un párrafo los hallazgos del análisis (Sección ??), las ventajas de su algoritmo (Sección ??) y aprendizajes del laboratorio. Evite introducir datos nuevos; apóyese en lo ya mostrado.

[Conclusiones concisas y respaldadas por las figuras/tablas del paper.]

[Párrafo sugerido: “En el análisis del kickstarter (Sección ??) observamos [...] (Tab. ??, Fig. ??). Nuestro algoritmo (Sección ??) redujo la demora media en un X% y evitó bucles mediante [...]. Como trabajo futuro proponemos [...].”]

Agradecimientos

Opcional: reconocimientos a la cátedra, compañeros o herramientas.

References

- [1] A. Varga and R. Hornig, “OMNeT++,” in *Proc. Int. Conf. Simulation Tools and Techniques (Simutools)*, 2008.
- [2] Cátedra de Redes y Sistemas Distribuidos, “Laboratorio 4: kickstarter OMNeT++,” FaMAF/UNC, 2026.
- [3] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2011.